

✓ Reto: Deserción de Empleados

Objetivo: Evaluar, limpiar, crear y seleccionar características del dataset `empleadosRET0.csv` para construir un conjunto de datos adecuado para un modelo de clasificación binaria que prediga la deserción (`Attrition`) de empleados.

✓ 1. Importación de librerías

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call `drive.mount("/content/drive", force_remount=True)`.

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler, LabelEncoder
from sklearn.decomposition import PCA
import warnings
warnings.filterwarnings('ignore')
```

✓ 2. Lectura del dataset

```
EmpleadosAttrition = pd.read_csv('/content/drive/MyDrive/Data_Science (Tec. Monterrey)/15_Ingenieria_de_caracteristicas/06_Reto01/empleadosRET0.csv')
print(f'Shape: {EmpleadosAttrition.shape}')
EmpleadosAttrition.head()
```

Shape: (400, 30)

	Age	BusinessTravel	Department	DistanceFromHome	Education	EducationField	EmployeeCount	EmployeeNumber	EnvironmentSatisfaction
0	50	Travel_Rarely	Research & Development	1 km	2	Medical	1	997	
1	36	Travel_Rarely	Research & Development	6 km	2	Medical	1	178	
2	21	Travel_Rarely	Sales	7 km	1	Marketing	1	1780	
3	52	Travel_Rarely	Research & Development	7 km	4	Life Sciences	1	1118	
4	33	Travel_Rarely	Research & Development	15 km	1	Medical	1	582	

5 rows × 30 columns

```
# Vista general del dataset
EmpleadosAttrition.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 400 entries, 0 to 399
Data columns (total 30 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Age                   400 non-null   int64
1   BusinessTravel        396 non-null   object
2   Department            400 non-null   object
3   DistanceFromHome      400 non-null   object
4   Education             400 non-null   int64
5   EducationField        400 non-null   object
6   EmployeeCount         400 non-null   int64
7   EmployeeNumber        400 non-null   int64
8   EnvironmentSatisfaction 400 non-null   int64
9   Gender               400 non-null   object
10  JobInvolvement        400 non-null   int64
11  JobLevel              400 non-null   int64
12  JobRole               400 non-null   object
13  JobSatisfaction       400 non-null   int64
14  MaritalStatus         395 non-null   object
15  MonthlyIncome         400 non-null   int64
```

```

16 NumCompaniesWorked      400 non-null  int64
17 HiringDate              400 non-null  object
18 Over18                  400 non-null  object
19 OverTime                 400 non-null  object
20 PercentSalaryHike       400 non-null  int64
21 PerformanceRating       400 non-null  int64
22 RelationshipSatisfaction 400 non-null  int64
23 StandardHours           400 non-null  int64
24 TotalWorkingYears       400 non-null  int64
25 TrainingTimesLastYear   400 non-null  int64
26 WorkLifeBalance         400 non-null  int64
27 YearsInCurrentRole      400 non-null  int64
28 YearsSinceLastPromotion 400 non-null  int64
29 Attrition               400 non-null  object
dtypes: int64(19), object(11)
memory usage: 93.9+ KB

```

3. Eliminación de columnas irrelevantes

Las siguientes columnas no aportan información para predecir la deserción porque tienen un único valor para todos los empleados o son identificadores únicos:

Columna	Razón
EmployeeCount	Todos tienen valor 1
EmployeeNumber	ID único por empleado (no informativo)
Over18	Todos tienen "Y"
StandardHours	Todos tienen 80

```

cols_to_drop = ['EmployeeCount', 'EmployeeNumber', 'Over18', 'StandardHours']
EmpleadosAttrition.drop(columns=cols_to_drop, inplace=True)

print(f'Columnas eliminadas: {cols_to_drop}')

```

```
Columnas eliminadas: ['EmployeeCount', 'EmployeeNumber', 'Over18', 'StandardHours']
```

4. Creación de la variable YearsAtCompany a partir de HiringDate

```

EmpleadosAttrition['Year'] = pd.to_datetime(EmpleadosAttrition['HiringDate'], format='mixed', dayfirst=True, errors='coerce').dt
EmpleadosAttrition['Year'] = EmpleadosAttrition['Year'].fillna(0).astype(int)
print('Primeros valores de Year:')
print(EmpleadosAttrition[['HiringDate', 'Year']].head(10))

```

```

Primeros valores de Year:
   HiringDate  Year
0  06/06/2013  2013
1  12/25/2015  2015
2   2/14/2017  2017
3   7/29/2010  2010
4  10/07/2011  2011
5  10/17/1996  1996
6   1/16/2016  2016
7  11/28/2012  2012
8   8/25/2006  2006
9   1/21/2012  2012

```

```

# Crear columna YearsAtCompany: años en la empresa hasta 2018
EmpleadosAttrition['YearsAtCompany'] = 2018 - EmpleadosAttrition['Year']

print('Estadísticas de YearsAtCompany:')
print(EmpleadosAttrition['YearsAtCompany'].describe())

```

```

Estadísticas de YearsAtCompany:
count      400.000000
mean       12.282500
std        100.716245
min         0.000000
25%         3.000000
50%         5.000000
75%        10.000000
max        2018.000000
Name: YearsAtCompany, dtype: float64

```

Se observa un valor maximo de experiencia de 2018, vamos a verificar donde esta el error:

```
EmpleadosAttrition[EmpleadosAttrition['Year']==0]['HiringDate']
```

```
HiringDate
229    2/30/2012
```

dtype: object

```
# Supondremos que es un error de captura de datos, ya que la fecha de contratacion la marca como el 30 de febrero de 2012, por e:
EmpleadosAttrition['Year'] = pd.to_datetime(EmpleadosAttrition['HiringDate'], format='mixed', dayfirst=True, errors='coerce').dt
EmpleadosAttrition.loc[(EmpleadosAttrition['Year'].isna()) & (EmpleadosAttrition['HiringDate'].astype(str).str.contains('2012'))

# Ahora sí, rellenar cualquier NaT restante con 0 y convertir a int
EmpleadosAttrition['Year'] = EmpleadosAttrition['Year'].fillna(0).astype(int)

print('Primeros valores de Year:')
print(EmpleadosAttrition[['HiringDate', 'Year']].head(10))

print(f"\nValor de 'HiringDate' en el índice 229: {EmpleadosAttrition.loc[229, 'HiringDate']}")
print(f"Valor de 'Year' en el índice 229: {EmpleadosAttrition.loc[229, 'Year']}")
```

Primeros valores de Year:

```
HiringDate  Year
0  06/06/2013  2013
1  12/25/2015  2015
2   2/14/2017  2017
3   7/29/2010  2010
4  10/07/2011  2011
5  10/17/1996  1996
6   1/16/2016  2016
7  11/28/2012  2012
8   8/25/2006  2006
9   1/21/2012  2012
```

Valor de 'HiringDate' en el índice 229: 2/30/2012

Valor de 'Year' en el índice 229: 2012

5. Corrección de la variable DistanceFromHome

La columna contiene valores como "1 km", "6 km", etc. Se debe convertir a entero.

```
# Renombrar DistanceFromHome → DistanceFromHome_km
EmpleadosAttrition.rename(columns={'DistanceFromHome': 'DistanceFromHome_km'}, inplace=True)
```

```
print('Ejemplos de DistanceFromHome_km:')
print(EmpleadosAttrition['DistanceFromHome_km'].head())
```

Ejemplos de DistanceFromHome_km:

```
0    1 km
1    6 km
2    7 km
3    7 km
4   15 km
```

Name: DistanceFromHome_km, dtype: object

```
# Crear nueva columna DistanceFromHome como entero (sin ' km')
EmpleadosAttrition['DistanceFromHome'] = (
    EmpleadosAttrition['DistanceFromHome_km']
    .astype(str)
    .str.replace(' km', '', regex=False)
    .str.strip()
    .astype(int)
)

print('Ejemplos de DistanceFromHome (entero):')
print(EmpleadosAttrition['DistanceFromHome'].head())
```

Ejemplos de DistanceFromHome (entero):

```
0    1
1    6
2    7
3    7
```

```
4 15
Name: DistanceFromHome, dtype: int64
```

6. Eliminación de columnas auxiliares

Una vez extraída la información necesaria, se eliminan `Year`, `HiringDate` y `DistanceFromHome_km`.

```
EmpleadosAttrition.drop(columns=['Year', 'HiringDate', 'DistanceFromHome_km'], inplace=True)

print('Columnas actuales:')
print(EmpleadosAttrition.columns.tolist())
```

```
Columnas actuales:
['Age', 'BusinessTravel', 'Department', 'Education', 'EducationField', 'EnvironmentSatisfaction', 'Gender', 'JobInvolvement', 'J
```

7. Sueldo promedio por departamento (tabla informativa)

Se genera un frame informativo con el `MonthlyIncome` promedio por departamento.

```
SueldoPromedioDepto = (
    EmpleadosAttrition.groupby('Department')['MonthlyIncome']
    .mean()
    .reset_index()
    .rename(columns={'MonthlyIncome': 'SueldoPromedio'})
)

print('Sueldo promedio por departamento:')
SueldoPromedioDepto
```

Sueldo promedio por departamento:

	Department	SueldoPromedio
0	Human Resources	6239.888889
1	Research & Development	6804.149813
2	Sales	7188.250000

Próximos pasos: [Generar código con SueldoPromedioDepto](#) [New interactive sheet](#)

8. Escalado de `MonthlyIncome` al rango [0, 1]

```
scaler = MinMaxScaler()
EmpleadosAttrition['MonthlyIncome'] = scaler.fit_transform(
    EmpleadosAttrition[['MonthlyIncome']]
)

print('Rango de MonthlyIncome después del escalado:')
print(f" Min: {EmpleadosAttrition['MonthlyIncome'].min():.4f}")
print(f" Max: {EmpleadosAttrition['MonthlyIncome'].max():.4f}")
```

```
Rango de MonthlyIncome después del escalado:
Min: 0.0000
Max: 1.0000
```

9. Conversión de variables categóricas a numéricas

Se aplica `LabelEncoder` a cada variable categórica restante.

```
categorical_cols = [
    'BusinessTravel', 'Department', 'EducationField',
    'Gender', 'JobRole', 'MaritalStatus', 'Attrition',
    'OverTime'
]

le = LabelEncoder()
for col in categorical_cols:
```

```

EmpleadosAttrition[col] = le.fit_transform(
    EmpleadosAttrition[col].astype(str)
)
print(f' {col}: {EmpleadosAttrition[col].unique()}')

print('\nTipos de datos tras la codificación:')
print(EmpleadosAttrition.dtypes)

```

```

BusinessTravel: [2 0 1 3]
Department: [1 2 0]
EducationField: [3 2 1 5 4 0]
Gender: [1 0]
JobRole: [5 4 8 0 3 7 2 6 1]
MaritalStatus: [0 2 1 3]
Attrition: [0 1]
OverTime: [0 1]

```

Tipos de datos tras la codificación:

```

Age                int64
BusinessTravel     int64
Department         int64
Education          int64
EducationField     int64
EnvironmentSatisfaction int64
Gender             int64
JobInvolvement     int64
JobLevel           int64
JobRole            int64
JobSatisfaction    int64
MaritalStatus      int64
MonthlyIncome      float64
NumCompaniesWorked int64
OverTime           int64
PercentSalaryHike  int64
PerformanceRating  int64
RelationshipSatisfaction int64
TotalWorkingYears  int64
TrainingTimesLastYear int64
WorkLifeBalance    int64
YearsInCurrentRole int64
YearsSinceLastPromotion int64
Attrition          int64
YearsAtCompany     int64
DistanceFromHome   int64
dtype: object

```

10. Selección de variables por correlación con Attrition

Se calcula la correlación lineal de cada variable con la variable objetivo y se conservan únicamente las que tienen $|\text{correlación}| \geq 0.1$ (incluyendo Attrition).

```

# Calcular correlación de cada variable con Attrition
correlaciones = EmpleadosAttrition.corr()['Attrition'].abs().sort_values(ascending=False)

print('Correlación (valor absoluto) con Attrition:')
print(correlaciones.to_string())

```

Correlación (valor absoluto) con Attrition:

```

Attrition          1.000000
OverTime           0.324777
JobLevel           0.214266
TotalWorkingYears  0.213329
Age                0.212121
YearsInCurrentRole 0.203918
MonthlyIncome      0.194936
MaritalStatus      0.187283
JobInvolvement     0.166785
JobSatisfaction    0.164957
EnvironmentSatisfaction 0.124327
JobRole            0.078684
TrainingTimesLastYear 0.070884
YearsSinceLastPromotion 0.069000
PercentSalaryHike  0.060880
BusinessTravel     0.060677
Education          0.055531
Department         0.054236
DistanceFromHome   0.052732
EducationField     0.051184
YearsAtCompany     0.032718
RelationshipSatisfaction 0.030945

```

Gender	0.028839
WorkLifeBalance	0.021723
NumCompaniesWorked	0.009082
PerformanceRating	0.006471

11. Seleccionar variables igual o mayores a correlacion 0.1

```
# Seleccionar variables con correlación >= 0.1 (incluye Attrition como salida)
cols_seleccionadas = correlaciones[correlaciones >= 0.1].index.tolist()

# Asegurar que Attrition esté en la selección
if 'Attrition' not in cols_seleccionadas:
    cols_seleccionadas.append('Attrition')

EmpleadosAttritionFinal = EmpleadosAttrition[cols_seleccionadas].copy()

print(f'Variables seleccionadas ({len(cols_seleccionadas)}): {cols_seleccionadas}')
EmpleadosAttritionFinal.head()
```

Variables seleccionadas (11): ['Attrition', 'OverTime', 'JobLevel', 'TotalWorkingYears', 'Age', 'YearsInCurrentRole', 'MonthlyIncome', 'MaritalStatus', 'JobInvolvement', 'JobSatisfaction']

	Attrition	OverTime	JobLevel	TotalWorkingYears	Age	YearsInCurrentRole	MonthlyIncome	MaritalStatus	JobInvolvement	JobSatisfaction
0	0	0	4	32	50	4	0.864269	0	3	3
1	0	0	2	7	36	2	0.207340	0	3	3
2	1	0	1	1	21	0	0.088062	2	3	3
3	0	0	3	18	52	6	0.497574	2	3	3
4	1	1	3	15	33	6	0.664470	1	3	3

Próximos pasos: [Generar código con EmpleadosAttritionFinal](#) [New interactive sheet](#)

12. Análisis de Componentes Principales (PCA)

Se aplica PCA sobre `EmpleadosAttritionFinal` y se determina el mínimo número de componentes que explican al menos el 80% de la varianza.

```
# Separar características (excluir Attrition para el PCA)
features_pca = [c for c in EmpleadosAttritionFinal.columns if c != 'Attrition']
X_pca = EmpleadosAttritionFinal[features_pca].values

# Aplicar PCA con todos los componentes posibles
pca = PCA()
EmpleadosAttritionPCA = pca.fit_transform(X_pca)

# Varianza explicada acumulada
varianza_acumulada = np.cumsum(pca.explained_variance_ratio_)

print('Varianza explicada por cada componente:')
for i, (indiv, acum) in enumerate(zip(pca.explained_variance_ratio_, varianza_acumulada)):
    print(f' PC{i} → {indiv:.4f} ({indiv*100:.2f}%) | Acumulada: {acum*100:.2f}%')
    if acum >= 0.80:
        print(f'\n✓ Con {i+1} componente(s) se alcanza el 80% de la varianza.')
        break
```

Varianza explicada por cada componente:
 PC0 → 0.7281 (72.81%) | Acumulada: 72.81%
 PC1 → 0.1812 (18.12%) | Acumulada: 90.93%

✓ Con 2 componente(s) se alcanza el 80% de la varianza.

```
# Determinar el número mínimo de componentes para >= 80% de varianza
n_components_80 = int(np.argmax(varianza_acumulada >= 0.80)) + 1
print(f'Número mínimo de componentes para 80% de varianza: {n_components_80}')
```

Número mínimo de componentes para 80% de varianza: 2

13. Agregar los componentes C1 y C2 al dataframe.

```
# Agregar los componentes principales como columnas C0, C1, ..., C(n-1)
for i in range(n_components_80):
    EmpleadosAttritionFinal = EmpleadosAttritionFinal.assign(
        **{f'C{i}': EmpleadosAttritionPCA[:, i]}
    )

print(f'Columnas de componentes agregadas: {[f"C{i}" for i in range(n_components_80)]}')
print(f'Shape final de EmpleadosAttritionFinal: {EmpleadosAttritionFinal.shape}')
EmpleadosAttritionFinal.head()
```

Columnas de componentes agregadas: ['C0', 'C1']
Shape final de EmpleadosAttritionFinal: (400, 13)

	Attrition	OverTime	JobLevel	TotalWorkingYears	Age	YearsInCurrentRole	MonthlyIncome	MaritalStatus	JobInvolvement	JobSatisfaction
0	0	0	4	32	50	4	0.864269	0	3	3
1	0	0	2	7	36	2	0.207340	0	3	3
2	1	0	1	1	21	0	0.088062	2	3	3
3	0	0	3	18	52	6	0.497574	2	3	3
4	1	1	3	15	33	6	0.664470	1	3	3

Próximos pasos:

[Generar código con EmpleadosAttritionFinal](#)

[New interactive sheet](#)

14. Exportación del dataset final a CSV

```
EmpleadosAttritionFinal.to_csv('EmpleadosAttritionFinal.csv', index=False)
print('Archivo EmpleadosAttritionFinal.csv guardado exitosamente.')
```

Archivo EmpleadosAttritionFinal.csv guardado exitosamente.